SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Artificial Intelligence and Data Science

ASSESSMENT TOOL 2 – SCENARIO BASED

Course Code:	DSA03	**Course Title:**	Natural Language Processing
CO Assessed:	CO2 – Manipulation	**Assessment:**	Assessment Tool 2 – SCENARIO BASED
Weightage:	35% of CIA		
CO5 Statement:	Design inflectional, derivational, finite-state parsing, stemming algorithms and for analyse morphological methods. (BL-3)		
Student Name:	Abinesh H	**Reg. No.:**	192424387
Section / Batch:	B.Tech AIDS / 3rd year	**Date:**	06/08/2026

Code of Conduct

I __Abinesh H (192424387)___ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____Abinesh H_____

Instructions

- Read all questions carefully before attempting the answers.
- Answer all five questions. Each question carries equal marks.
- Show all intermediate steps, calculations, probability computations, tagging decisions, and justifications wherever applicable.
- Duration: 30 minutes.

Section / Topic	Focus	Q Nos
English Word Classes, Tagsets for English, Stochastic Part of Speech Tagging, HMM	Morphological parsing of disagree, agreement, and agreeable; identification of prefixes, suffixes, base forms, and semantic effects of derivation.	Q1

Annexure A – SCENARIO BASED

A company is developing an NLP-based grammar checker for educational applications. The system is trained using a manually POS-tagged corpus. Your task is to recreate the Hidden Markov Model (HMM) **without using any built-in NLP or HMM libraries**.

From the given corpus,

- determine the **Emission Probability**
- determine the **Transition Probability**
- implement the **Viterbi Algorithm**
- predict the POS tags for the new input sentence.

Use only Python dictionaries, loops, and basic programming constructs.

Training Corpus

Sentence	POS Tags
The/DT boy/NN eats/VBZ rice/NN	DT NN VBZ NN
The/DT girl/NN drinks/VBZ milk/NN	DT NN VBZ NN
A/DT cat/NN drinks/VBZ milk/NN	DT NN VBZ NN
The/DT dog/NN chases/VBZ cat/NN	DT NN VBZ NN
A/DT teacher/NN teaches/VBZ students/NNS	DT NN VBZ NNS
Students/NNS study/VBP English/NN	NNS VBP NN
Birds/NNS fly/VBP high/RB	NNS VBP RB
Children/NNS play/VBP games/NNS	NNS VBP NNS

New Input Sentence

The cat drinks milk

Tasks

1. Separate each sentence into words and POS tags.
2. Compute the **Emission Probability** of every word.
3. Compute the **Transition Probability** between POS tags.
4. Recreate the Hidden Markov Model using Python dictionaries.
5. Implement the **Viterbi Algorithm** without using any built-in HMM/NLP libraries.
6. Predict the POS tag sequence for “The cat drinks milk”

Rubrics for Calculation

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Understanding of Scenario	20%	Demonstrates clear and complete understanding of the scenario and context	Shows good understanding with minor gaps	Partial understanding; misses key aspects	Misinterprets or fails to understand the scenario
Identification of Key Issues	20%	Accurately identifies all relevant issues and factors involved	Identifies most key issues with minor omissions	Identifies limited issues; some irrelevant points	Unable to identify key issues
Application of Concepts	25%	Applies appropriate concepts effectively to address the scenario	Applies concepts with minor errors	Limited or partially correct application	Unable to apply relevant concepts
Decision Making	20%	Makes well-reasoned, logical decisions with strong rationale	Makes reasonable decisions with some justification	Makes weak decisions with limited justification	No clear decisions made or rationale provided
Clarity of Response	15%	Response is clear, structured, and logically presented	Generally clear with minor issues	Somewhat unclear or poorly structured	Disorganized and difficult to understand

Q. No. / Criteria Level	Understanding of Scenario	Identification of Key Issues	Application of Concepts	Decision Making	Clarity of Response	Sub. Total	Total %
1	4	4	4	3	3	18	92
2	4	4	3	4	4	19	
3	3	4	4	4	4	18	
4	4	4	3	4	4	19	
5	4	4	4	4	3	18	

Faculty In-charge	Course Coordinator	Program Director
Dr. Kumaragurubaran T		
Signature: _____	Signature: _____	Signature: _____
Date: 06/08/2026	Date: _____	Date: _____

Hidden Markov Model (HMM)

Code:

```
from collections import defaultdict
corpus = [
    ("The boy eats rice".split(), ["DT","NN","VBZ","NN"]),
    ("The girl drinks milk".split(), ["DT","NN","VBZ","NN"]),
    ("A cat drinks milk".split(), ["DT","NN","VBZ","NN"]),
    ("The dog chases cat".split(), ["DT","NN","VBZ","NN"]),
    ("A teacher teaches students".split(), ["DT","NN","VBZ","NNS"]),
    ("Students study English".split(), ["NNS","VBP","NN"]),
    ("Birds fly high".split(), ["NNS","VBP","RB"]),
    ("Children play games".split(), ["NNS","VBP","NNS"])
]
print("TASK 1 : Words and POS Tags\n")

for words, tags in corpus:
    print("Words :", words)
    print("Tags  :", tags)
    print()

emission_count = defaultdict(lambda: defaultdict(int))
transition_count = defaultdict(lambda: defaultdict(int))
tag_count = defaultdict(int)
start_count = defaultdict(int)

for words, tags in corpus:

    start_count[tags[0]] += 1

    for w, t in zip(words, tags):
        emission_count[t][w.lower()] += 1
        tag_count[t] += 1

    for i in range(len(tags)-1):
        transition_count[tags[i]][tags[i+1]] += 1

print("TASK 2 : Emission Probability")

emission_prob = defaultdict(dict)

for tag in emission_count:
    for word in emission_count[tag]:
        emission_prob[tag][word] = emission_count[tag][word] / tag_count[tag]

for tag in emission_prob:
    print(tag)
    for word in emission_prob[tag]:
        print(f"{word:10s} : {emission_prob[tag][word]:.3f}")
    print()
```

```
print("TASK 3 : Transition Probability")

transition_prob = defaultdict(dict)

for tag in transition_count:
    total = sum(transition_count[tag].values())
    for nxt in transition_count[tag]:
        transition_prob[tag][nxt] = transition_count[tag][nxt] / total

for tag in transition_prob:
    print(tag)
    for nxt in transition_prob[tag]:
        print(f"{tag} --> {nxt} = {transition_prob[tag][nxt]:.3f}")
    print()

print("TASK 4 : Hidden Markov Model")

print("\nStates:")
states = list(tag_count.keys())
print(states)

print("\nStart Probability")

start_prob = {}

total_sent = len(corpus)

for tag in states:
    start_prob[tag] = start_count[tag]/total_sent
    print(tag, "=", round(start_prob[tag],3))

def emission(tag, word):
    return emission_prob[tag].get(word.lower(),0.0001)

def transition(tag1, tag2):
    return transition_prob[tag1].get(tag2,0.0001)

def start(tag):
    return start_prob.get(tag,0.0001)

def viterbi(sentence):

    V=[{}]
    path={}

    for state in states:
        V[0][state]=start(state)*emission(state,sentence[0])
        path[state]=[state]

    for t in range(1,len(sentence)):
        V.append({})
        new_path={}

        for curr in states:
```

```
            best_prob=-1
            best_state=None

            for prev in states:

                prob=V[t-1][prev]*transition(prev,curr)*emission(curr,sentence[t])

                if prob>best_prob:
                    best_prob=prob
                    best_state=prev

            V[t][curr]=best_prob
            new_path[curr]=path[best_state]+[curr]

        path=new_path

    max_prob=-1
    best=None

    for state in states:
        if V[-1][state]>max_prob:
            max_prob=V[-1][state]
            best=state

    return path[best]

print("TASK 6 : POS Prediction")

sentence="The cat drinks milk".split()

pred=viterbi(sentence)

print("Sentence :"," ".join(sentence))
print("Predicted POS Tags :",pred)
```

Output:

```
Words and POS Tags

Words : ['The', 'boy', 'eats', 'rice']
Tags  : ['DT', 'NN', 'VBZ', 'NN']

Words : ['The', 'girl', 'drinks', 'milk']
Tags  : ['DT', 'NN', 'VBZ', 'NN']

Words : ['A', 'cat', 'drinks', 'milk']
Tags  : ['DT', 'NN', 'VBZ', 'NN']

Words : ['The', 'dog', 'chases', 'cat']
Tags  : ['DT', 'NN', 'VBZ', 'NN']

Words : ['A', 'teacher', 'teaches', 'students']
Tags  : ['DT', 'NN', 'VBZ', 'NNS']

Words : ['Students', 'study', 'English']
Tags  : ['NNS', 'VBP', 'NN']

Words : ['Birds', 'fly', 'high']
Tags  : ['NNS', 'VBP', 'RB']

Words : ['Children', 'play', 'games']
Tags  : ['NNS', 'VBP', 'NNS']
```

```
Emission Probability
DT
the         : 0.600
a           : 0.400

NN
boy         : 0.100
rice        : 0.100
girl        : 0.100
milk        : 0.200
cat         : 0.200
dog         : 0.100
teacher     : 0.100
english     : 0.100

VBZ
eats        : 0.200
drinks      : 0.400
chases      : 0.200
teaches     : 0.200
```

```
NNS
students    : 0.400
birds       : 0.200
children    : 0.200
games       : 0.200

VBP
study       : 0.333
fly         : 0.333
play        : 0.333

RB
high        : 1.000
```

```
Transition Probability
DT
DT --> NN = 1.000

NN
NN --> VBZ = 1.000

VBZ
VBZ --> NN = 0.800
VBZ --> NNS = 0.200

NNS
NNS --> VBP = 1.000

VBP
VBP --> NN = 0.333
VBP --> RB = 0.333
VBP --> NNS = 0.333
```

```
Hidden Markov Model

States:
['DT', 'NN', 'VBZ', 'NNS', 'VBP', 'RB']

Start Probability
DT = 0.625
NN = 0.0
VBZ = 0.0
NNS = 0.375
VBP = 0.0
RB = 0.0
POS Prediction
Sentence : The cat drinks milk
Predicted POS Tags : ['DT', 'NN', 'VBZ', 'NN']
```